

EosDashboards

Production API and UI installation on IIS

A step-by-step guide for a Windows Server hosting multiple IIS sites and APIs

This guide covers the API publish folder, the UI dist folder, secrets, database migration, IIS setup, verification, and rollback.

This guide installs EosDashboards on a Windows Server that already hosts many other IIS websites and APIs. It keeps EosDashboards isolated through unique bindings, separate application pools, versioned release folders, and separate deployment credentials.

Values inside `<...>` are placeholders. Replace them with values approved by the infrastructure team. Never commit real passwords, tokens, connection strings, private keys, certificates, or personal data.

1. Target IIS layout

Use a dedicated IIS site, or a dedicated application area approved by the infrastructure team. The recommended layout is one site with two applications:

IIS item	Example value	What it contains
Site	EosDashboardsSite	Only this product
HTTPS binding	https://dashboards.<company-domain>	A unique host name and certificate
UI application	/EosDashboards	The static React output
API application	/EosDashboardsApi	The ASP.NET Core publish output
UI pool	EosDashboardsUiPool	Static files only
API pool	EosDashboardsApiPool	The ASP.NET Core process
Deployment identity	EosDashboardsDeploy	Release copy and database migration

More than one IIS site may listen on TCP 443. The HTTPS binding must have a unique host name/SNI and the correct certificate. Never reuse another product's site, application pool, physical path, or certificate without approval.

2. Get the production decisions first

Before changing IIS, obtain these values from the infrastructure and database owners:

1. The final DNS name and HTTPS host name.
2. The certificate and its renewal owner.
3. The IIS site, application, and pool names assigned to EosDashboards.
4. The API Windows/service identity and the separate deployment identity.
5. The API release folder and ACL policy for appsettings.Production.json.
6. The SQL Server and database name.
7. The approved backup and restore procedure.
8. The installed ASP.NET Core Hosting Bundle version.
9. The release directory and log directory permissions.
10. The person who approves migrations and rollback.

Do not begin by editing the existing sites. First inventory their bindings, application pools, certificates, paths, and identities so the new values are provably unique.

3. Build the API release

Build outside the production server, from one committed source revision. Open PowerShell in the repository root, where backend and frontend exist:

```
Set-Location D:\Workspaces\ChatGpt\EosDashboards
```

```
$releaseId = "20260910-120000"

$apiArtifact = ".\artifacts\release\$releaseId\api"
$uiArtifact = ".\artifacts\release\$releaseId\ui"

New-Item -ItemType Directory -Force $apiArtifact | Out-Null
New-Item -ItemType Directory -Force $uiArtifact | Out-Null

dotnet publish `
.\backend\src\EosDashboards.Api\EosDashboards.Api.csproj `
-c Release `
-o $apiArtifact
```

The API output is the complete folder in \$apiArtifact. Copy the whole folder, not only the DLL. A valid API artifact normally contains:

```
EosDashboards.Api.dll
web.config
appsettings.json
*.deps.json
*.runtimeconfig.json
referenced DLL files
```

The API artifact does not contain index.html or the React assets directory. The API web.config must remain with the artifact because it starts the ASP.NET Core process and removes IIS WebDAV interception for API verbs such as PUT and DELETE.

4. Build and collect the UI release

The UI output is different from the API output. It is a static Vite directory, not a DLL and not the source frontend/src directory.

From the same repository root and the same \$releaseId:

```
npm ci --prefix frontend

$env:VITE_PUBLIC_BASE = "/EosDashboards/"
$env:VITE_API_BASE_URL = "/EosDashboardsApi"

npm --prefix frontend run build

Copy-Item -Path .\frontend\dist\* -Destination $uiArtifact -Recurse -Force
```

The folder to copy to the server is the complete contents of:

```
frontend\dist\
```

The UI artifact must contain at least:

```
index.html
assets\...
web.config
```

Do not copy `frontend\src`, `frontend\public`, `node_modules`, or the entire frontend repository to IIS. Do not copy only `index.html`; the `assets` folder and `web.config` are required.

If the approved production topology uses different application paths or separate host names, build with those exact values instead:

```
$env:VITE_PUBLIC_BASE = "/"
$env:VITE_API_BASE_URL = "https://api-<company-domain>"
npm --prefix frontend run build
```

The UI build values are public browser values. They must not contain passwords, database credentials, JWT signing keys, SMS credentials, or other secrets. The API must also be configured with the exact browser origin for CORS, such as `https://dashboards.<company-domain>`; a path is not part of an origin.

5. Build the migration artifact

Build the Migration Bundle from the same committed revision as the API and UI:

```
New-Item -ItemType Directory -Force ".\artifacts\release\<releaseId>\migration" | Out-Null

dotnet ef migrations bundle `
--self-contained `
--target-runtime win-x64 `
--configuration Release `
--project .\backend\src\EosDashboards.Infrastructure\EosDashboards.Infrastructure.csproj `
--startup-project .\backend\src\EosDashboards.Api\EosDashboards.Api.csproj `
--output ".\artifacts\release\<releaseId>\migration\EosDashboards.Migrations.exe"
```

Building the bundle does not change a database. The database is selected only when the executable is run on the deployment server with that database's connection string.

6. Package and transfer the release

Transfer these three versioned directories using the approved artifact transfer method:

```
artifacts\release\<release-id>\api\
artifacts\release\<release-id>\ui\
artifacts\release\<release-id>\migration\EosDashboards.Migrations.exe
```

On the server, use separate versioned directories:

```
D:\IIS\EosDashboards\Api\releases\<release-id>\
D:\IIS\EosDashboards\Ui\releases\<release-id>\
D:\IIS\EosDashboards\Deployments\<release-id>\
C:\ProgramData\EosDashboards\keys\
```

Example copy commands, to be run by the approved release process:

```
robocopy .\api D:\IIS\EosDashboards\Api\releases\<release-id> /E /COPY:DAT /R:2 /W:5
robocopy .\ui D:\IIS\EosDashboards\Ui\releases\<release-id> /E /COPY:DAT /R:2 /W:5
robocopy .\migration D:\IIS\EosDashboards\Deployments\<release-id> /E /COPY:DAT /R:2 /W:5
```

For robocopy, exit codes 0 through 7 are normally success or success with differences; 8 or higher is a copy failure. Keep the previous release folders for rollback. Do not overwrite the active release in place.

7. Where each secret belongs

The API reads these configuration sections. The exact production values must be in the server-side appsettings.Production.json file with a restrictive ACL. They must not be in Git or the browser-delivered UI artifact.

Configuration key	Used by	Store it in	Never put it in
ConnectionStrings:EosDashboard	API and migration runner	Server-side appsettings.Production.json	UI, Git, logs
AuthSecurity:HashingKey	API password/recovery security	Server-side appsettings.Production.json	UI or source control
AuthSecurity:SigningKey	API JWT signing	Server-side appsettings.Production.json	UI, logs, Git
AuthSecurity:Issuer and Audience	API token validation	Server-side appsettings.Production.json	UI secrets
AuthSecurity:KeyRingPath	API Data Protection	appsettings.Production.json plus a stable directory outside web root	Temporary release folder
Sms:Endpoint and Sms:Timeout	API SMS provider	Server-side appsettings.Production.json	Frontend build variables
GoogleAuthentication:ClientSecret	API Google OIDC, if enabled	Server-side appsettings.Production.json	UI or Git
ApiSecurity:AllowedOrigins	API CORS	Production API configuration	Wildcard origin

The production API configuration must be separate from development appsettings.Development.json. Put the complete production configuration in the server-side appsettings.Production.json file. A safe conceptual configuration looks like this:

```

ConnectionStrings__EosDashboard=<company-database-connection>
AuthSecurity__HashingKey=<long-random-secret>
AuthSecurity__SigningKey=<long-random-secret>
AuthSecurity__Issuer=<production-issuer>
AuthSecurity__Audience=<production-audience>
AuthSecurity__KeyRingPath=C:\ProgramData\EosDashboards\keys
Sms__Endpoint=<company-sms-endpoint>
ApiSecurity__AllowedOrigins__0=https://dashboards.<company-domain>

```

The double underscore form above is only the ASP.NET Core environment-variable equivalent; it is not required for this deployment. The JSON appsettings file is the source of runtime configuration and must contain the same keys and sections shown in the tracked template.

The repository contains the complete configuration template at backend/src/EosDashboards.Api/appsettings.Production.template.json. Before starting the API, create the real appsettings.Production.json beside EosDashboards.Api.dll in the API release directory, fill it with approved production values, and apply a restrictive ACL for the API identity and the deployment identity. The real file is ignored by Git; the template is the file that belongs in Git. For every new release, copy or recreate this server-side file in the new API release directory without printing its contents.

Set the API process environment to Production and verify that it is not Development; this makes ASP.NET Core load appsettings.Production.json. The file must be next to the published API files unless the application is explicitly extended with another configuration provider.

The migration runner needs only the database connection at execution time. The API runtime identity must not receive schema-change permission. The deployment identity may run reviewed migrations, but it is a different identity.

8. Prepare IIS without disturbing other sites

1. Install or verify the matching ASP.NET Core Hosting Bundle.
2. Install the approved HTTPS certificate in the server certificate store.
3. Create or verify the unique DNS record and HTTPS binding.
4. Create EosDashboardsUiPool and EosDashboardsApiPool.
5. Set both pools to ****No Managed Code**** and ****Integrated**** pipeline mode.
6. Set the API pool to the dedicated API identity, not Administrator.
7. Give the UI pool read/execute access to the UI release folders only.
8. Give the API pool read/execute access to API releases and write access only to the approved key-ring and log locations.
9. Create the UI application /EosDashboards pointing to the new UI release.
10. Create the API application /EosDashboardsApi pointing to the new API release.
11. Assign each application to its matching pool.
12. Keep the API and UI web.config files from the inspected artifacts.

The UI web.config must match the UI application path because its IIS 404 fallback returns to /EosDashboards/. If the organization chooses a different path, rebuild the UI with the new VITE_PUBLIC_BASE and update the tested fallback configuration before deployment.

9. Backup, migrate, and activate in that order

1. Take and verify the approved database backup.
2. Open the deployment PowerShell in the migration directory:

```
Set-Location D:\IIS\EosDashboards\Deployments\<release-id>
```

3. Load the database connection from the API's server-side appsettings file without printing it or putting it in a source file:

```
$apiConfigPath = "D:\IIS\EosDashboards\Api\releases\<release-id>\appsettings.Production.json"
$apiConfig = Get-Content $apiConfigPath -Raw | ConvertFrom-Json
$dbConnection = $apiConfig.ConnectionStrings.EosDashboard
```

4. Run the migration bundle without echoing the command:

```
& .\EosDashboards.Migrations.exe --connection $dbConnection
```

5. Clear the in-memory variables after the migration:

```
Remove-Variable dbConnection,apiConfig,apiConfigPath
```

6. Verify that the command completed successfully and that the expected latest migration exists in __EFMigrationsHistory.

7. Only after migration success, point the API application to the new API release and start the API pool.

8. Point the UI application to the new UI release and start the UI pool.

If migration fails, do not activate the new API. The API does not apply schema migrations at startup. A normal API rollback does not automatically undo a successful database migration, so destructive schema changes require a planned expand/contract rollout and a DBA-approved recovery path.

10. Verify the actual HTTPS installation

Check these endpoints using the final host name:

```
https://<host>/EosDashboardsApi/health/live
https://<host>/EosDashboardsApi/health/ready
https://<host>/EosDashboards/
```

Each expected response must be HTTP 200. Also verify a direct browser refresh of an internal UI route. With an approved test account, check login, SMS OTP, one authorized page, session refresh, logout, and one safe read/mutation flow.

Check IIS logs, Windows Event Viewer, and API logs for startup or database errors. Never record passwords, OTPs, tokens, connection strings, or private keys in the deployment report.

11. Rollback

If the new release fails:

1. Stop further rollout and record the trace/error information without secrets.
2. Point the API application back to the previous API release.
3. Point the UI application back to the previous UI release.
4. Restart the affected pools in a controlled way.
5. Repeat health checks and the minimum smoke test.
6. If the database migration already succeeded, keep the previous API only if it

is compatible with the new schema. Otherwise use the approved forward-fix or database recovery procedure; do not run an unreviewed Down migration.

12. Final checklist

- [] Unique DNS host name, HTTPS binding, SNI, and certificate approved.
- [] No existing Site, Application, Pool, or physical path is reused.
- [] API publish output copied as a complete folder.
- [] UI dist output copied as a complete folder, including index.html, assets, and web.config.
- [] API, UI, and Migration Bundle came from the same commit.
- [] Production values are in server-side appsettings.Production.json and absent from Git and UI.
- [] Data Protection Key Ring is stable and outside the web root.
- [] Database backup is verified.
- [] Migration succeeded with the deployment identity.
- [] API and UI use separate pools with least-privilege identities.
- [] Health, UI, route-refresh, login, OTP, refresh, and logout checks passed.
- [] Previous release folders remain available for rollback.